

上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

人工智能实践教学中心
Artificial Intelligence Practice Teaching Center

基于yolo深度学习

蓝敏源
brucelan1777@sjtu.edu.cn

饮水思源 · 爱国荣校

01

yolo环境配置

您可以在此处添加您需要的文字，您可以在此处添加您需要的。

Anaconda环境配置

我们先配置Anaconda环境，熟悉Anaconda Prompt，学习怎么通过指令创建虚拟环境从而管理yolo环境。

1.下载安装Anaconda。如果你之前安装过Anaconda，但是不确定是否安装的有问题那么建议你先去控制面板卸载之前安装的Anaconda因为如果Anaconda版本过旧或者有其他问题可能导致后续出错卸载之前建议先确认有没有需要保留的环境。

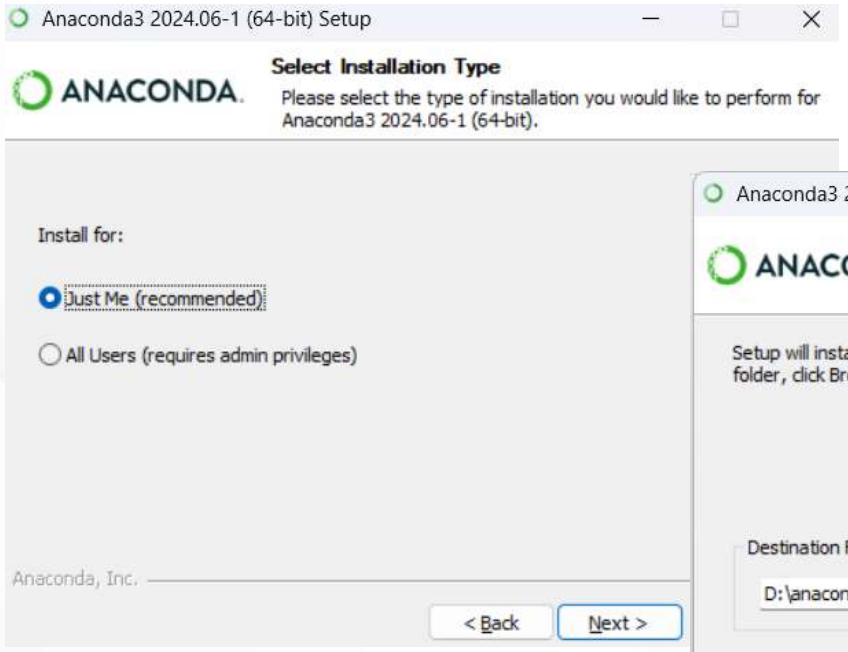
2.浏览器输入tuna anaconda进入清华大学开源软件镜像站，复制安装包网址并进入，就可以看到所有版本的anaconda。这里我们示例选用2024.06-1的版本，选择那个带Windows的直接下载。

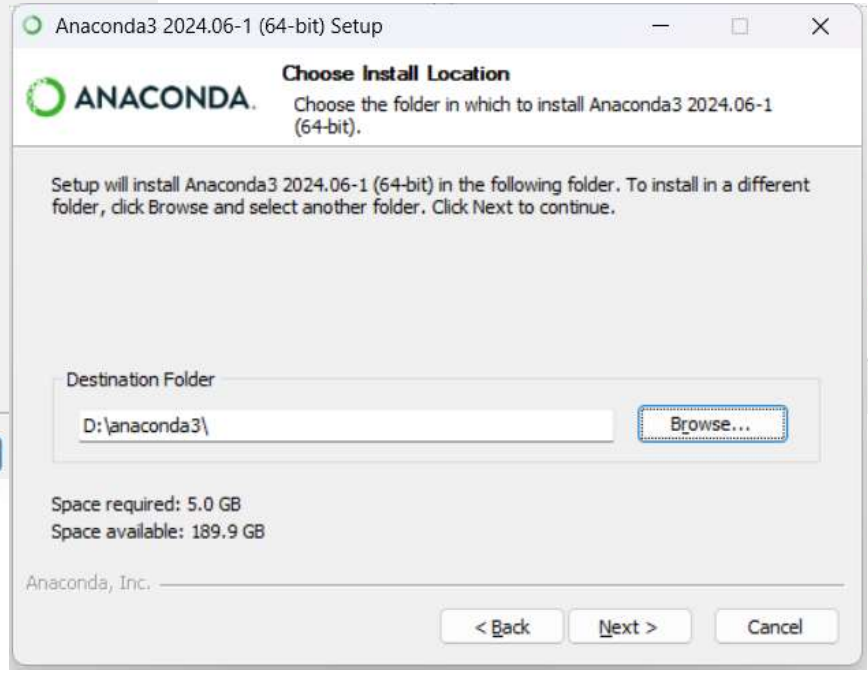


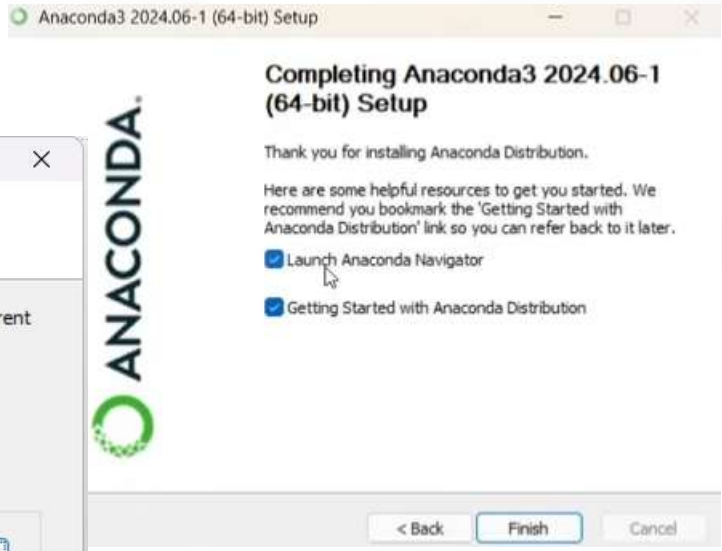


Anaconda环境配置

安装这一步时选择Just Me,不然会默认装到C盘导致C盘爆满，还会在后续的操作中因为有中文路径报错，所以自己安排路径创建一个空的文件夹切记不能有中文和空格。最后一步这两个勾取消，其余安装保持默认。





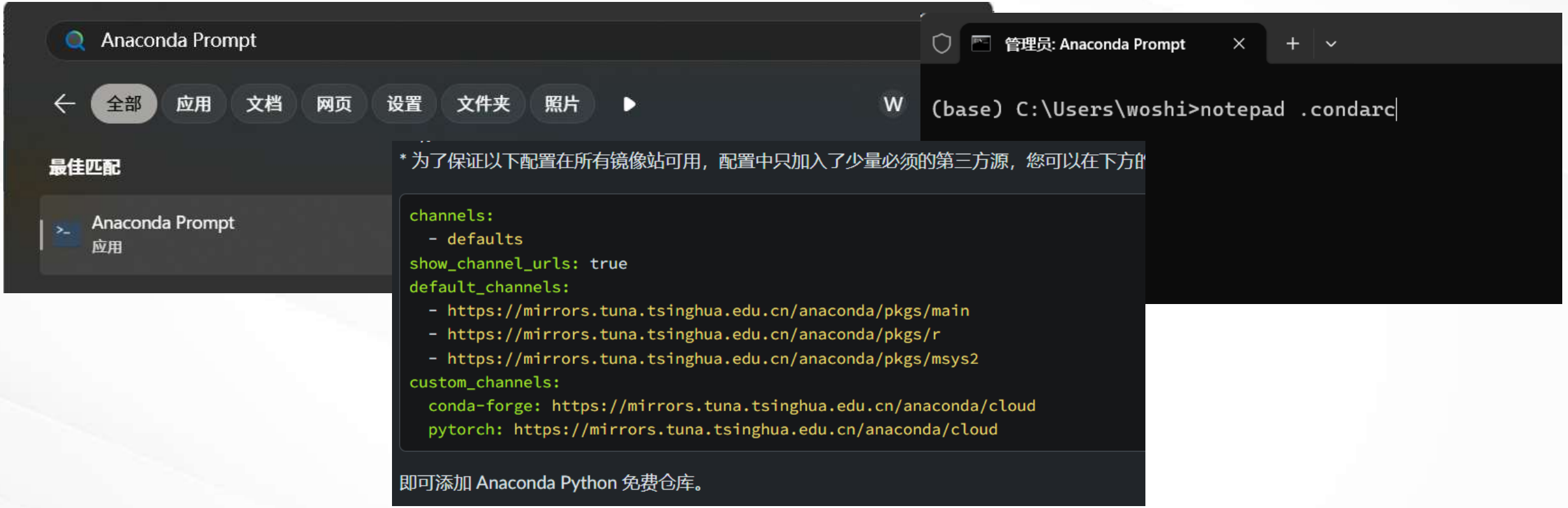


Anaconda环境配置

搜索栏搜索Anaconda Prompt点击进入不报错说明安装成功，输入notepad .condarc,输入刚刚清华大学开源网站下的第三方源链接保存退出。然后在终端输入以下指令

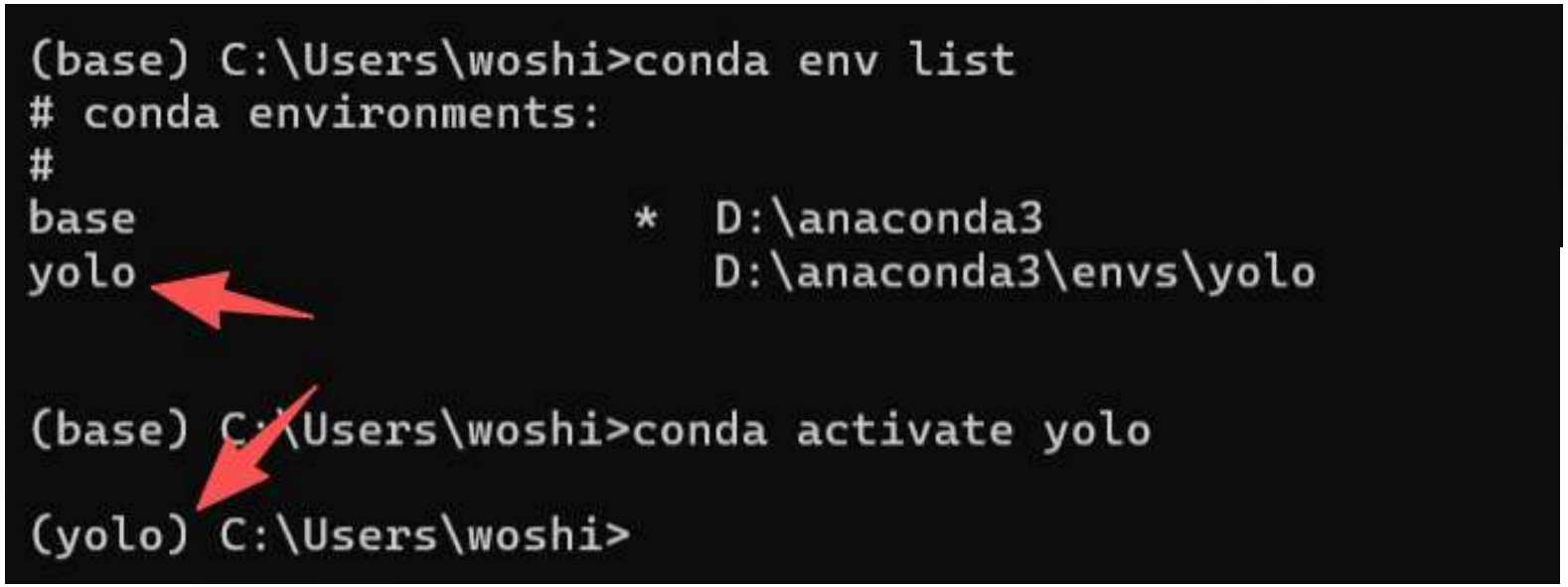
pip config set global.index-url https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple

如果输出Writing to C:\Users\woshi\AppData\Roaming\pip\pip.ini 说明你已经配置好了Anaconda。



Anaconda环境配置

进入Anaconda Prompt输入指令conda create -n yolo python=3.11，开始创建yolo虚拟环境，过程中遇到选项全部回车即可。安装完成后输入指令conda env list，检查列表是否存在yolo环境，然后再输入指令conda activate yolo,显示yolo标志说明已经成功进入yolo虚拟环境中了。



记住该指令conda activate +环境名称，每次打开之后默认是base需要输入指令激活指定的虚拟环境。

Anaconda环境配置

Wheel

OSX

```
pip install torch==2.5.0 torchvision==0.20.0 torchaudio==2.5.0
```

LINUX AND WINDOWS

```
# ROCM 6.1 (Linux only)
pip install torch==2.5.0 torchvision==0.20.0 torchaudio==2.5.0 --index-url https://download
# ROCM 6.2 (Linux only)
pip install torch==2.5.0 torchvision==0.20.0 torchaudio==2.5.0 --index-url https://download
# CUDA 11.8
pip install torch==2.5.0 torchvision==0.20.0 torchaudio==2.5.0 --index-url https://download
# CUDA 12.1
pip install torch==2.5.0 torchvision==0.20.0 torchaudio==2.5.0 --index-url https://download
# CUDA 12.4
pip install torch==2.5.0 torchvision==0.20.0 torchaudio==2.5.0 --index-url https://download
# CPU only
pip install torch==2.5.0 torchvision==0.20.0 torchaudio==2.5.0 --index-url https://download
```

浏览器输入https://pytorch.org/get-started/previous-versions/找到2.5.0版本。然后复制指令在yolo环境下运行，开始安装pytorch。安装成功后环境配置部分就成功了。

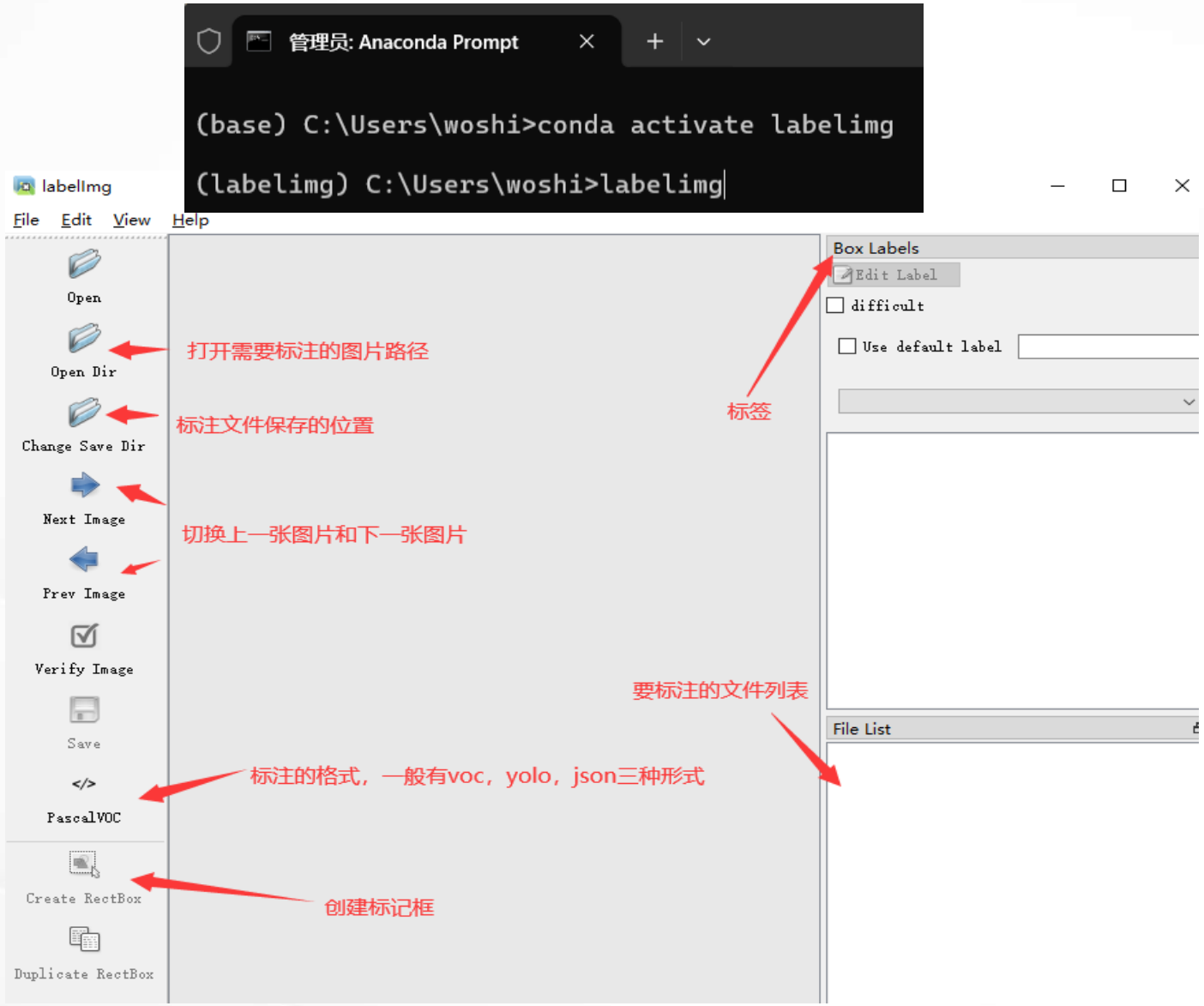


02

使用Labelimg制作YOLO标签

您可以在此处添加您需要的文字，您可以在此处添加您需要的。

认知Labelimg



labelimg是一款开源的图像标注工具，标签可用于分类和目标检测，它是用python写的，并使用Qt作为其图形界面，简单好用（虽然是英文版的）。参考第一节中的内容我们再创建一个labelimg的虚拟环境并激活，然后输入指令pip install labelimg -i https://pypi.tuna.tsinghua.edu.cn/simple 开始运行安装labelimg工具，完成后输入labelimg即可打开工具

图片收集

使用第四课时学习的视频录制代码采集视频。该代码需要在本地运行或者使用MobaXterm远程链接工具。

步骤4: 录制视频

编写代码以录制视频并保存：

录制设定时间的视频并保存到xgoVideos目录下

Python | 运行代码 | 复制代码

```python
from picamera2 import Picamera2
import time
import cv2 as cv
import numpy as np
import os
from datetime import datetime
from xgolib import XGO
dog = XGO("xgomini")
dog.attitude('p', -12)
class Dog_Camera(object):
    """狗狗相机类 - 自动录制10秒后退出"""
    def __init__(self, width=640, height=480, debug=False):
        self.__debug = debug
        self.__state = False
        self.__width = width
        self.__height = height

        # 新增录制相关属性
        self.__is_recording = False
        self.__video_writer = None
        self.__video_path = "/home/pi/xgoVideos/"
        self.__record_start_time = 0  # 录制开始时间
        self.__record_duration = 10    # 固定录制时长：10秒

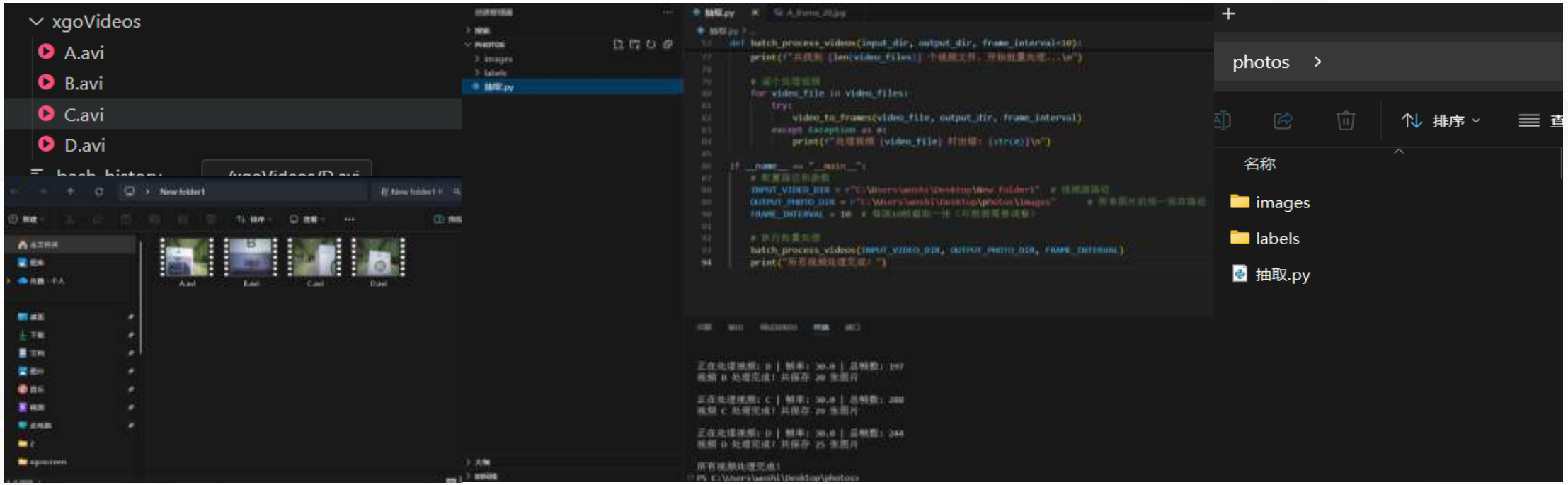
        print("---------Camera Init...-------------")
        # 初始化相机
        self.__picam2 = Picamera2()
        config = self.__picam2.create_preview_configuration(main={"format": 'RGB888', "si
        self.__picam2.configure(config)
        self.__picam2.start()
        self.__state = True

```

图片收集及预处理

选择录制视频作为原材料是因为视频录制时可以移动设备对目标进行多角度，多位置拍摄提高我们后续训练模型的泛化性。将小狗录制的视频下载出来保存到一个文件夹里，使用附件提供的抽取脚本从视频里抽取照片到指定路径（创建一个images文件夹照片提取保存到这里方便后续标注）。

如下图所示，根据自己的实际路径修改路径参数然后运行脚本，成功后记得查看抽取的照片，删掉没有标记目标的不合格照片，再创建一个labels的文件夹作为保存标签的文件夹，最后页面如右下图所示。



图片标注

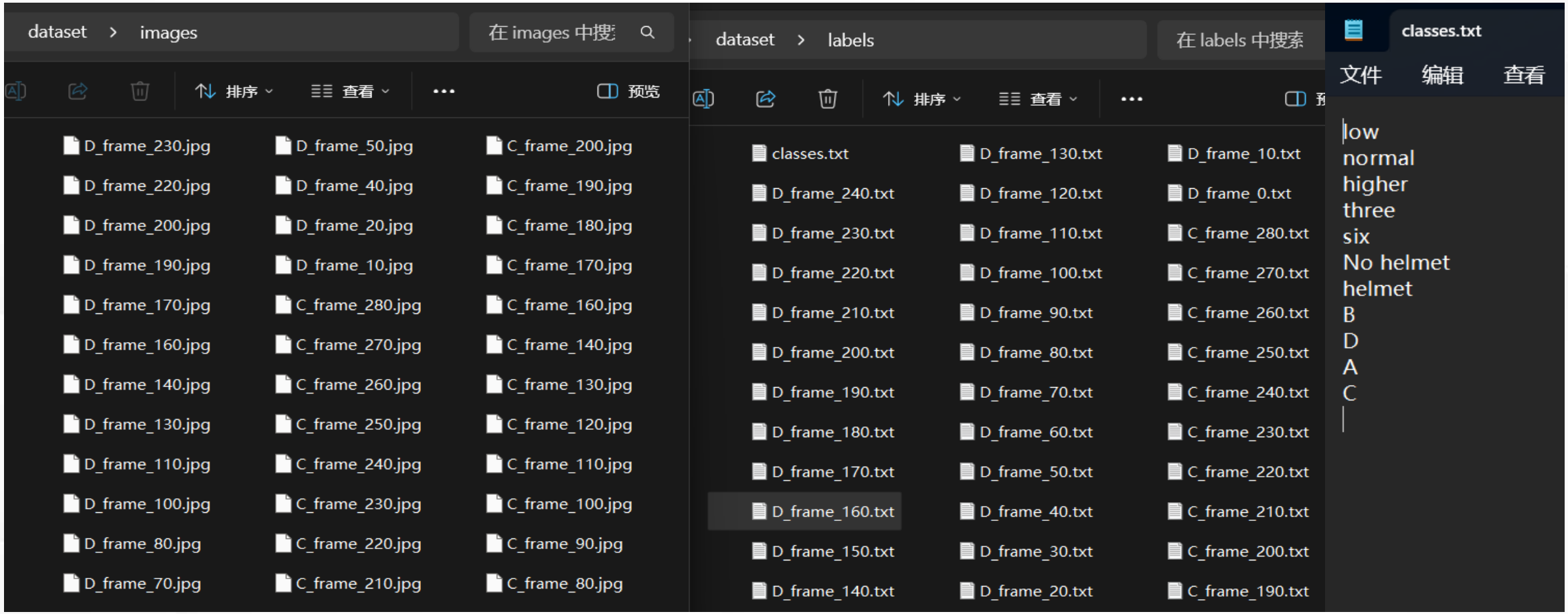
接下来我们要启动Labelimg手动给刚刚收集到的图片打标签。在这之前我们要先明确我们这一步要生成的三样东西。1.图片集；2.classes.txt文件；3.yolo标签文件；classes.txt是labelimg可视化标签用的必须与yolo标签文件在同一文件夹里。

我们先进入Anaconda Prompt然后再激活labelimg的虚拟环境，输入指令labelimg进入工作页面。



图片标注

全部完成之后就如下图所示，从左往右依次是图片集，yolo标签集，classes.txt文件。图片与标签一一对应，classes.txt里就是我们标注的类别序列，这样就完成了yolo标签的制作。



03

制作数据集及高效训练



您可以在此处添加您需要的文字，您可以在此处添加您需要的。

制作数据集

数据集分为训练集（train），验证集（val）,测试集（test）（测试集可选可不选），本节课演示只保留训练集和验证集。

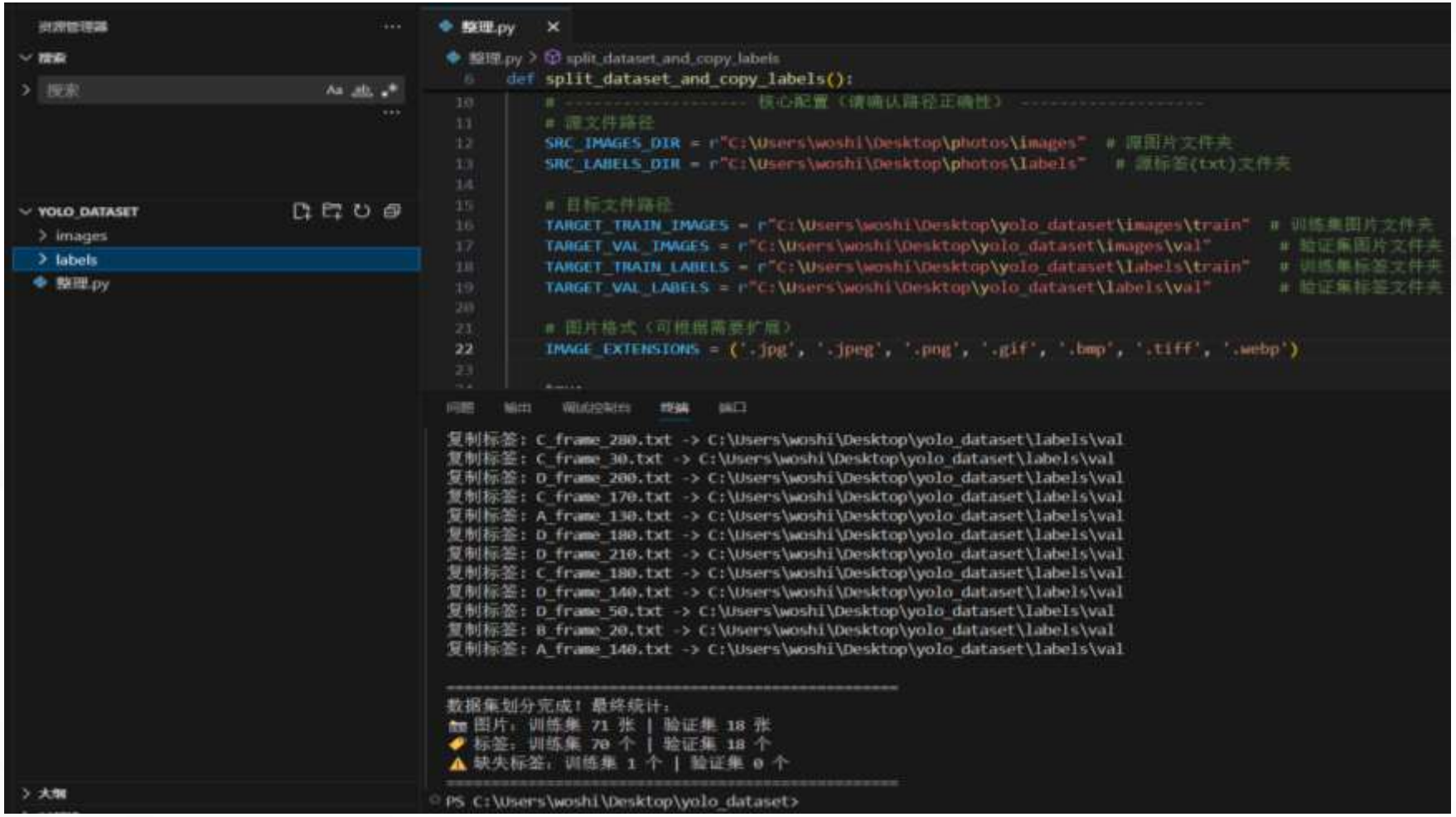
我们先创建一个yolo_dataset的文件夹再创建各类子文件夹如下图所示来存放我们上节课做好的图片集和标签集。

图片与标签必须一一对应否则接下来的训练中就会报错，你也可以直接使用附件里的整理脚本一键整理。

```
yolo_dataset/
├── images/
│   ├── train/--------------存放80%的图片
│   └── val/----------------存放20%的图片
└── labels/
    ├── train/--------------存放那80%的图片的标签
    └── val/----------------存放那20%的图片的标签
```

制作数据集

如果我们图片集有很多的话就可以使用这个脚本，只需要把源文件和目标文件的路径改为自己的实际路径即可，但是运行成功后我们需要去打开文件夹挨个检查一下尽量保证每个类别都能按比例随机分布在train和val里。



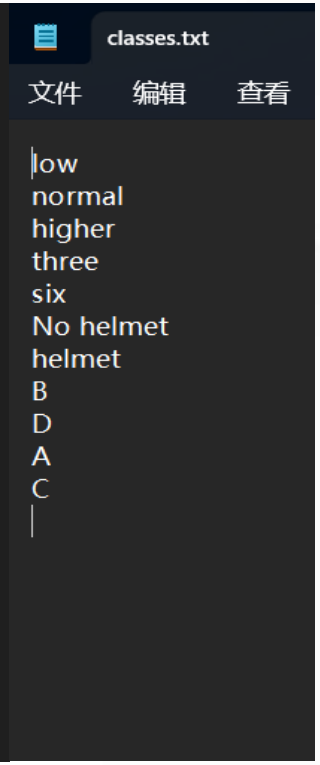
配置yaml文件

接下来我们就要配置yaml文件，yaml文件相当于指路牌，在训练的时候告诉我们的yolo模型文件所需要的路径和目标对应的类别。

下图所示就是yaml文件的内容格式是通用的，把路径改成自己的即可。names的内容就是使用Labelimg标签时产生的classes.txt的内容，复制过来，顺序不可以出错否则就会导致后续训练的类别混乱。

```yaml
# 数据集配置文件（关键：路径和类别要对应）
path: C:\Users\woshi\Desktop\yolo_dataset  # 划分后的数据集根目录（绝对路径/相对路径都可以）
train: C:\Users\woshi\Desktop\yolo_dataset\images\train       # 训练集图片路径（相对于path）
val: C:\Users\woshi\Desktop\yolo_dataset\images\val           # 验证集图片路径

names:
  0: low
  1: normal
  2: higher
  3: three
  4: six
  5: No helmet
  6: helmet
  7: B
  8: D
  9: A
  10: C
```



训练

打开附件中的mytrain.py文件如下所示，这里我们选用yolo11n.pt模型进行训练，

1.修改yaml路径为你自己的实际路径。

2.epochs：训练轮数开始越多模型越准确，最后准确度趋于一定值不再变化，故轮数不是越大越好，建议默认。

3.imgsz:缩放倍数，训练时先将图片缩放成640*640的正方形，建议设置成原图尺寸即可。

4.batch：将训练集分批次处理的批量，硬件条件会对值有所限制设为-1抛给yolo对你的设备测试，得出适合的最优值。

5.device：是否使用GPU处理，0使用GPU处理（效率翻倍），1使用CPU。

6.save：是否保存结果。

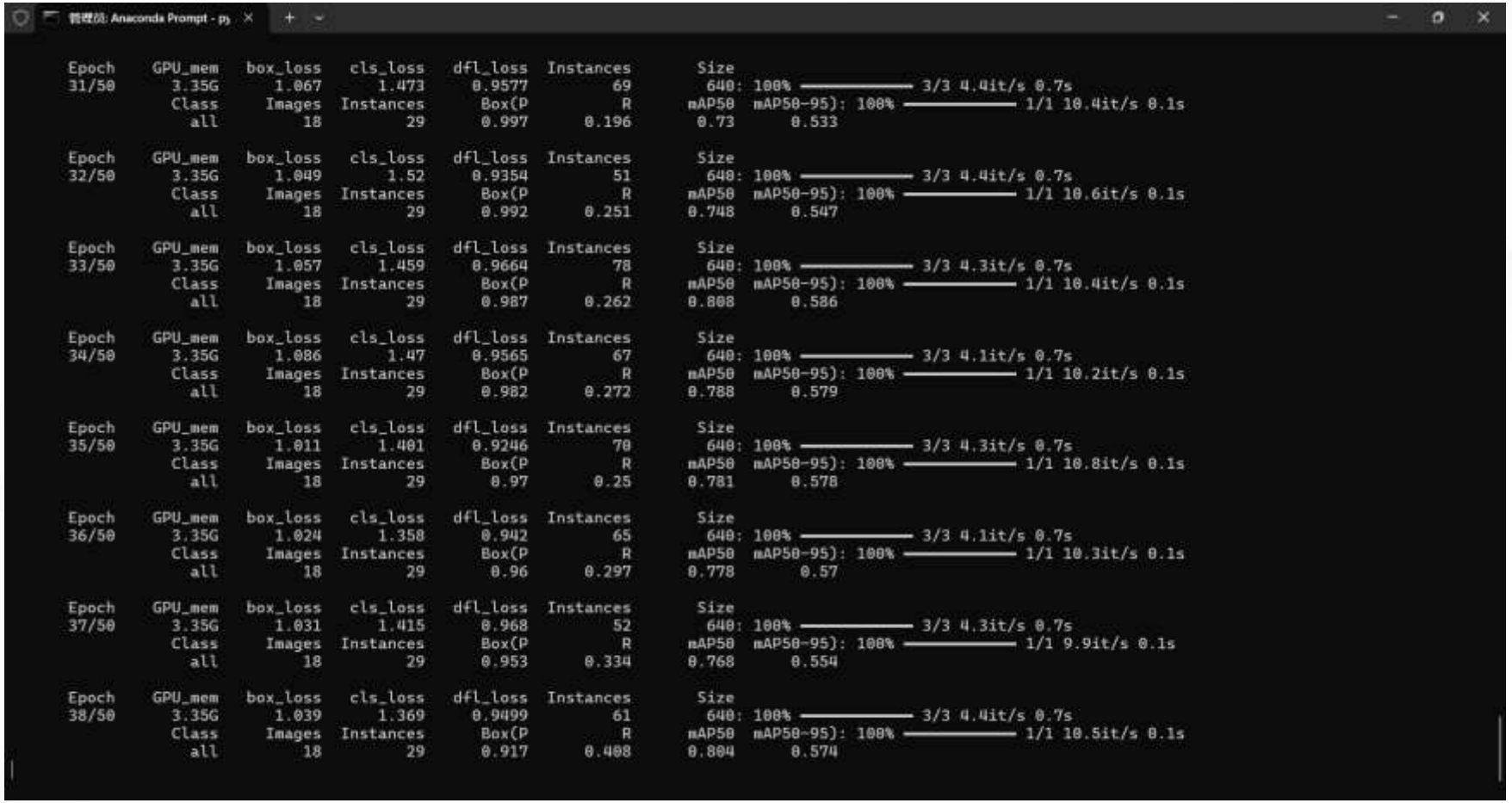
mytrain.py > ...

```python
from ultralytics import YOLO
if __name__ == "__main__":
    model = YOLO(r"yolo11n.pt")  #预训练模型或路径
    model.train(
        data=r"C:\Users\woshi\Desktop\yolo_dataset\yolo_dataset.yaml",  #yaml文件路径
        epochs=50,
        imgsz=640,
        batch=-1,
        device=0,
        save=True
        )
```

训练

进入Anaconda Prompt，激活yolo环境输入python +mytrain.py路径运行mytrain.py。

终端会实时反馈训练状态，等待训练结束即可。



训练

最后会产生两个文件，一个是训练效果最好的的模型文件，一个是最后一次训练的模型文件，一般选择best.pt。到这一步训练就结束了，保存模型文件的路径。

```
50 epochs completed in 0.030 hours.
Optimizer stripped from C:\Users\woshi\runs\detect\train13\weights\last.pt, 5.5MB
Optimizer stripped from C:\Users\woshi\runs\detect\train13\weights\best.pt, 5.5MB

Validating C:\Users\woshi\runs\detect\train13\weights\best.pt...
Ultralytics 8.3.252  Python-3.11.14 torch-2.5.0+cu118 CUDA:0 (NVIDIA GeForce RTX 4050 Laptop GPU, 6140MiB)
YOLO11n summary (fused): 100 layers, 2,584,297 parameters, 0 gradients, 6.3 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 1/1 4.4it/s 0.2s
                   all         18         29      0.938      0.642      0.881      0.615
                normal          5          5          1      0.598      0.995      0.542
                higher          2          2          1          0     0.0828     0.0565
                 three          1          1          1          0      0.995      0.497
                   six          3          3          1      0.547      0.995      0.548
                     B          3          3      0.782          1      0.995      0.852
                     D          7          7       0.95          1      0.995      0.767
                     A          3          3      0.772          1      0.995      0.786
                     C          5          5          1       0.99      0.995      0.876
Speed: 0.2ms preprocess, 4.6ms inference, 0.0ms loss, 1.7ms postprocess per image
Results saved to C:\Users\woshi\runs\detect\train13

(yolo) C:\Users\woshi>
```

最后会产生最优的和最后一次的两个模型文件，复制路径并保存

名称	修改日期	类型
best.pt	2026/1/16 17:06	PT 文件
last.pt	2026/1/16 17:06	PT 文件